RAPORT

Testowanie modeli Whisper i Bielik na wypowiedziach matematycznych w języku polskim – kalkulator głosowy

Wydział: Inżynierii Mechanicznej i Robotyki

Kierunek: Inżynieria Akustyczna

Rok: 3

Przedmiot: Technologia mowy

Grupa: 3

Imię i nazwisko:

Mateusz Juszczak – *Team Leader*

Oliwia Chmura

Henryk Kosała

1. Cel projektu

Celem projektu jest zaprojektowanie i implementacja systemu do automatycznej analizy poprawności modeli rozpoznawania mowy (ASR) oraz przetwarzania języka naturalnego (NLP) na przykładzie wypowiedzi matematycznych w języku polskim. Projekt koncentruje się na ocenie jakości transkrypcji generowanych przez model Whisper oraz analizie wpływu błędów ASR na dalszą interpretację treści matematycznych przez model językowy Bielik. System umożliwia ilościową ocenę skuteczności modeli poprzez porównanie transkrypcji z danymi referencyjnymi oraz analizę stabilności interpretacji równań matematycznych w obecności błędów rozpoznawania mowy.

2. Architektura systemu

2.1. Dane

2.1.1. Dane referencyjne

Do realizacji projektu wykorzystano polskojęzyczny zbiór nagrań utworzonych przez członków grupy. Zbiór ten zawiera nagrania działań matematycznych wraz z odpowiadającymi im transkrypcjami referencyjnymi oraz poprawnymi wynikami obliczeń. Dzięki tak przygotowanym danym, możliwa jest interpretacja poprawności otrzymanych wyników oraz analiza potencjalnych błędów na poszczególnych etapach przetwarzania. Zgromadzone dane dotyczą działań w zakresie liczb rzeczywistych i koncentrują się na wybranych działaniach: dodawanie, odejmowanie, dzielenie, mnożenie, potęgowanie, pierwiastkowanie, obliczenia procentowe.

Dane są przechowywane na zewnętrznym dysku Google i wczytywane do systemu za pomocą interfejsu Google API.

2.1.2. Dane pobierane przez zewnętrzny mikrofon

W ramach programu użytkownik ma możliwość wprowadzić dane w czasie rzeczywistym za pomocą zewnętrznego mikrofonu. Wypowiedź użytkownika, która zawiera

nagranie danego działania matematycznego jest rejestrowana w postaci audio i jest bezpośrednio przekazywana do modułu rozpoznawania mowy.

2.2. Model rozpoznawania mowy – Whisper

Model Whisper pełni rolę systemu automatycznego rozpoznawania mowy (ASR), przekształcającego sygnał audio w reprezentację tekstową. W projekcie wykorzystano gotowe warianty modelu Whisper, które umożliwiają rozpoznawanie mowy w języku polskim bez dodatkowego treningu. Wynikiem działania modelu jest transkrypcja nagrania audio, która następnie porównywana jest z transkrypcją referencyjną.

2.3. Analiza treści matematycznych – model Bielik

Transkrypcje wygenerowane przez model Whisper stanowią wejście dla modelu językowego Bielik, który odpowiada za interpretację treści matematycznej wypowiedzi. Model Bielik wykorzystywany jest do:

- rozpoznawania struktury działania matematycznego,
- obliczania wyniku działania,

Dla celów porównawczych model Bielik testowany jest również na transkrypcjach referencyjnych (ground truth), co pozwala ocenić bezpośredni wpływ błędów ASR na jakość przetwarzania NLP.

2.4. Ewaluacja

Ewaluacja systemu obejmuje dwa poziomy analizy:

1. Ocena systemu ASR (Whisper):
   - analiza typowych błędów w rozpoznawaniu liczb i operatorów.
2. Ocena NLP (Bielik):
   - skuteczność interpretacji działań matematycznych,
   - porównanie wyników uzyskanych na ground truth oraz na transkrypcjach ASR,
   - analiza propagacji błędów ASR do etapu NLP.

Dodatkowo możliwe jest przeprowadzenie analizy jakościowej poprzez ręczne przeglądy błędnych przypadków.

2.5. Optymalizacja architektury i metodyka PAL

W ramach prac implementacyjnych przetestowano zoptymalizowane podejście hybrydowe, dostosowane do specyfiki sprzętowej (GPU AMD). W badaniach testowano różne modele Whisper oraz model językowy Bielik-7B poddany 4-bitowej kwantyzacji w celu optymalizacji zużycia pamięci VRAM.

Wdrożono podział obciążenia obliczeniowego, w którym transkrypcja ASR realizowana jest na procesorze (CPU) przy użyciu biblioteki *faster-whisper* z kwantyzacją int8, natomiast model językowy uruchamiany jest na GPU w środowisku Ollama. Dodatkowo zmodyfikowano logikę działania modułu NLP, stosując podejście *Program-Aided Language Models* (PAL). W tym wariancie model Bielik nie wykonuje obliczeń bezpośrednio (co eliminuje ryzyko halucynacji numerycznych), lecz pełni funkcję parsera semantycznego, tłumaczącego język naturalny na wyrażenie w języku Python, którego wynik obliczany jest w sposób deterministyczny.

3. Plan kamieni milowych

3.1. Kamień milowy 1 – MVP

- Stworzenie bazy danych.
- Uruchomienie modelu Whisper i generacja transkrypcji.
- Wstępne testy modelu Bielik na transkrypcjach referencyjnych i ASR.

3.2. Kamień milowy 2 – Analiza wpływu błędów ASR

- Porównanie wyników Bielika dla ground truth i transkrypcji Whispera.
- Analiza przypadków, w których błędy ASR prowadzą do błędnej interpretacji działań.
- Kategoryzacja błędów (liczby, operatory, struktura równania).
- Wizualizacja wyników (przykłady błędów).

3.3. Kamień milowy 3 – Optymalizacja i raport końcowy

- Testy różnych wariantów modelu Whisper oraz Bielik
- Moduł pobierania danych w czasie rzeczywistym
- Stworzenie GUI
- Optymalizacja pipeline'u (batch processing, caching).
- Przygotowanie raportu końcowego wraz z wykresami, tabelami oraz wnioskami

4. Status pracy

4.1. Status na dzień 17.12.2025 r.

Wykonane zadania okresie pracy do 17.12.2025 r.:

1. Przygotowanie bazy testowej nagrań.
2. Podział programu na moduły:

- dataset_download, gdzie:
    a) Konfiguracja środowiska i uwierzytelnienie połączenia API
    b) Pobieranie zbioru danych z dataset common voice
    c) Przygotowanie danych: rozpakowanie archiwum, walidacja danych i zapis do pliku
- program, gdzie:
    a) Implementacja wczytywania i obróbki danych podawanych na wejściu – usuwanie znaków, zamiana wielkości liter, lematyzacja tekstu: funkcja polish_text_to_math_expression(), flush_number()
    b) Konfiguracja i użycie modelu Whisper.
    c) Połączenie z modelem Bielik.

4.2. Status na dzień 7.01.2026 r.

Wykonane zadania okresie pracy do 7.01.2026 r.:

1. Powiększenie bazy testowej nagrań wraz z plikiem csv zawierającym referencyjną treść nagrania i poprawny wynik.

2. Utworzenie modułu wczytywania danych w czasie rzeczywistym z zewnętrznego mikrofonu.
3. Zaimplementowanie wczytywania danych z zewnętrznego dysku – moduł download_from_drive()
4. Stworzenie algorytmu, umożliwiającego zapisywanie na bieżąco historii dokonywanych zmian w konfiguracji modelu.
5. Usunięcie zbędnych funkcjonalności związanych z Common Voice.
6. Utworzenie interfejsu GUI do funkcjonalności wczytywania danych z zewnętrznego mikrofonu.
7. Analiza błędów związanych z rozpoznawaniem działań matematycznych
8. Analiza różnych konfiguracji modeli i otrzymywanych rezultatów.

5. Analiza błędów

5.1. Analiza błędów związanych z rozpoznawaniem działań matematycznych

5.1.1. Błędy fonetyczne warstwy akustycznej (ASR)

W początkowej fazie Whisper wykazywał systematyczne problemy z polskimi liczebnikami, interpretując słowa o podobnym brzmieniu jako zupełnie inne wyrazy (np. „trzy” rozpoznawane jako „krzy”, „cztery” jako „pisztery”, „czterdzieści” jako „cieści”). Te zniekształcenia uniemożliwiały modelowi językowemu (Bielik) poprawną analizę równania.

5.1.2. Błędy interpretacji i formatowania (Bielik)

Druga grupa błędów wynikała z ograniczeń modelu językowego przy stosowaniu sztywnych instrukcji. Model często „halucynował”, generując tekstowe opisy zamiast kodu lub stosując symbole matematyczne niezgodne ze składnią Pythona (np. używał `√` zamiast `math.sqrt` lub `^` zamiast `**`). Prowadziło to do błędów wykonania programu, nawet gdy sama transkrypcja mowy była poprawna.

5.1.3. Problemy z transkrypcją liczb złożonych:

System wykazywał istotne trudności przy obsłudze dużych liczb (np. „tysiąc dwieście trzydzieści trzy”). Model akustyczny miał tendencję do zapisywania ich słownie (robił przeważnie błędnie) lub przy próbie wymuszenia zapisu cyfrowego generował niezrozumiałe hybrydy i błędy fonetyczne (np. „dwie chcie” zamiast „200”), których model językowy nie potrafił poprawnie „skleić” w jedną wartość.

5.2. Analiza różnych konfiguracji modeli i otrzymywanych rezultatów

5.2.1. Konfiguracja bazowa

W punkcie wyjścia (Whisper tiny + Bielik z szczegółowym system promptem) system osiągnął skuteczność około 40%, ale jego skuteczność malała wraz z dodawaniem kolejnych poprawek i przy dużych system promptach zmalała do 20%.

5.2.2. Optymalizacja programowa (Few-Shot Learning)

W drugiej fazie zmieniono strategię promptowania na uczenie na przykładach (user prompt) oraz zaimplementowano w kodzie automatyczne korygowanie niestandardowych symboli matematycznych, których nie obsługiwała funkcja eval(). Pozwoliło to podnieść wynik do 65%. Błędy przeważnie wynikały z błędnego rozpoznania przez ASR.

5.2.3. Zmiana modelu akustycznego

W ramach testu sprawdzono czy większy model Whispera (base) jest w stanie wyeliminować problemy modelu tiny. Zgodnie z oczekiwaniami poprawiła się jego skuteczność do 80%. Dalej jednak w niektórych przypadkach system ma problemy z dużymi liczbami oraz niektórymi wyrażeniami ułamkowymi.